

DSA STUDY BLUEPRINT SERIES

# 70. The Celebrity Problem Stack Queue

Finding the Celebrity in a Group

Section 08 • C++ Core Data Structures & Algorithms

# Core Concepts & Visual Blueprint

Theoretical models and memory architectures

---

## Key Principles

- **Celebrity:** Knows no one, but everyone knows them.
- Use two-pointer elimination: candidate is *A* if *A* knows *B* (eliminate *A*) else eliminate *B*.

## Memory & Execution Blueprint

Candidate *A* knows *B*  $\Rightarrow$  *A* eliminated • *B* remains candidate.

# C++ Implementation Reference

Standard code layout and syntax

---

```
int getCelebrity(int n) {  
    int s = 0, e = n - 1;  
    while (s < e) {  
        if (knows(s, e)) s++; else e--;  
    }  
    return check(s) ? s : -1;  
}
```

# Algorithmic Complexity & Best Practices

Performance evaluations and critical guidelines

## Performance Table

| Aspect / Case    | Complexity |
|------------------|------------|
| Time Complexity  | $O(N)$     |
| Space Complexity | $O(1)$     |

## Key Practices & Gotchas

- **Important:** Always validate the final candidate to ensure the celebrity condition is met.
- Verify boundary conditions (e.g. empty inputs, single elements).
- Optimize dynamic memory allocations to prevent leaks.

# Dry Run & Step-by-Step Execution

Trace analysis on a sample input case

## Execution Walkthrough

Let's trace monotonic stack modifications for next greater element on `[4, 5, 2]` :

| Active Element | Stack State (Indices) | Pop comparison      | Result output            |
|----------------|-----------------------|---------------------|--------------------------|
| 4              | [0] (value 4)         | N/A                 | Waiting for next greater |
| 5              | [1] (value 5)         | 5 > 4 (pop index 0) | Next Greater of 4 is 5   |

# Interview Variations & Optimization Pathways

Common follow-up problems and optimization strategies

---

## Key Variations & Follow-up Questions

- **Monotonic properties:** Stacks process elements strictly increasing or decreasing, optimizing nested lookups.
- **Min Stack details:** Track current minimum bounds by storing elements as encoded difference values.
- **Related LeetCode Problems:** #155 Min Stack, #84 Largest Rectangle in Histogram, #42 Trapping Rain Water.